

Unidad 4 — Prototipado de proyectos de IA

Validar rápido y barato antes de invertir · "Toma de decisiones basadas en datos con IA" · Máster en IA

Esta unidad cambia de registro respecto a las anteriores: no introduce una familia de algoritmos ni matemática nueva (de hecho, no contiene una sola fórmula), sino una **disciplina de gestión de riesgos** para proyectos de aprendizaje automático. La pregunta de fondo es brutalmente práctica: *antes de gastar seis meses y un presupuesto serio construyendo un modelo, ¿cómo averiguo en dos o tres semanas si la idea sirve?* La respuesta del curso es el **prototipo**, entendido de una forma muy específica que conviene no confundir con lo que "prototipo" significa en otros contextos.

Dónde encaja en el curso: en la unidad anterior aprendiste a **elegir** qué iniciativas de IA perseguir con la matriz de **casos campeones** (alto impacto × alta viabilidad). Esta unidad es el paso siguiente y simétrico: una vez elegida una apuesta, el prototipo **confirma esa apuesta a bajo costo** antes de comprometer recursos. Verás que las dos dimensiones que el prototipo valida —impacto y viabilidad— son exactamente los dos ejes de aquella matriz. Es la misma brújula, aplicada un paso más adelante.

ANTES DE EMPEZAR — UNA NOTA SOBRE EL TÍTULO

El título oficial de la unidad es "Análisis de diagnóstico con IA", pero el contenido completo trata de **prototipado**, no de analítica diagnóstica. Es un desajuste del rótulo del curso: el tema real, y lo que se evalúa, es todo lo que sigue sobre prototipos. No te confundas si en el examen el nombre de la unidad no calza con lo que estudiaste.

1 Conceptos clave y propósito de la unidad

El objetivo declarado es usar la creación de prototipos como técnica para **gestionar la incertidumbre** inherente a los proyectos de ML, validando rápido las suposiciones de **viabilidad** e **impacto** antes de comprometer grandes inversiones. La mentalidad que se busca instalar tiene nombre propio: **"hablar rápido y barato"** *fail fast & cheap* — poner versiones iniciales del modelo frente a usuarios reales lo antes posible para obtener retroalimentación temprana, de modo que la solución final no solo sea técnicamente precisa, sino útil y aceptada por la organización.

El curso abre con seis conceptos clave que conviene fijar antes de avanzar, porque se usan en todo lo demás.

Incertidumbre uncertainty

Es la **falta de previsibilidad** sobre los requisitos, la aceptación del usuario y la calidad de los datos que rodea a casi todo proyecto de ML. No es ignorancia que se resuelva pensando más: es una propiedad estructural del problema. En software tradicional puedes especificar de antemano qué hará el sistema; en ML **no sabes si los datos contienen la señal necesaria hasta que construyes el modelo**. Esa imposibilidad de saber por adelantado es la justificación de toda la unidad: si pudieras predecir el resultado, no necesitarías prototipar. El curso la llama, sin pudor, "la increíble incertidumbre".

Prototipo prototype

Un **producto inacabado cuyo propósito principal y único es la validación** de hipótesis de negocio y técnicas. Esta es la definición canónica del curso y merece su propia sección (§2), porque su sentido aquí difiere del uso habitual de la palabra. Reténgalo así por ahora: no es "una versión chica del sistema", es **un instrumento para responder una pregunta y luego desecharse o escalar**.

Integración vertical profunda deep vertical integration

El enfoque de prototipado que **cubre todas las capas del caso de uso —desde la ingesta de datos hasta la interfaz de usuario— manteniendo baja la complejidad técnica**. La analogía precisa es la del **"esqueleto que camina"** (*walking skeleton*): en vez de construir una sola pieza perfecta y profunda (el modelo más exacto del mundo, pero sin nada alrededor), construyes un **hilo delgado pero completo** que conecta de punta a punta: un poco de datos reales → un modelo simple → una pantalla básica. Ese hilo delgado ya "camina" (funciona de extremo a extremo) y por eso permite validar. Es el corazón conceptual de la unidad y reaparece una y otra vez.

Instancia de cálculo compute instance

El recurso de **máquina virtual en la nube donde se ejecuta la carga de trabajo real** de los modelos: entrenar, evaluar, servir predicciones. Es el "motor" que arriendas por horas en la nube en lugar de comprar un servidor. En un prototipo te interesa que sea barato y de encendido/apagado rápido, para experimentar sin un compromiso de infraestructura.

Azure Blob Storage

El servicio de **almacenamiento de objetos binarios grandes** (*Blob = Binary Large Object*) de Microsoft Azure, usado para guardar tablas CSV, imágenes y resultados de modelos. Es el ejemplo concreto que el curso usa para la "capa de datos" (§4). Su equivalente en AWS es Amazon S3 y en Google Cloud es Cloud Storage; los tres hacen lo mismo: una bodega elástica donde dejas archivos de cualquier tipo y tamaño.

Azure Machine Learning Studio

La **interfaz web que ofrece herramientas para entrenar e implementar modelos sin necesidad de escribir código**. Es el ejemplo del curso para la "capa de análisis". Aquí hay un matiz de actualidad importante que se detalla en las notas de discrepancia al final.

NOTA — DISCREPANCIA 2 AZURE ML STUDIO

El producto que históricamente encajaba con esa descripción —arrastrar y soltar, sin código— era "**Azure ML Studio (classic)**", **retirado por Microsoft el 31 de agosto de 2024** [confirmado]. Hoy "Azure Machine Learning studio" es un entorno unificado **primordialmente orientado a código**, y la experiencia visual sin código la entrega el componente "**Azure ML designer**". Para el examen, la definición del curso ("interfaz web para entrenar/implementar sin código") es la que se evalúa; en la práctica de hoy, eso lo da el *designer*, y el *Studio classic* ya no existe.

2 Qué es un prototipo y por qué importa

2.1 La definición técnica del curso

En este curso, un prototipo **no es** simplemente una versión pequeña de un sistema. Se define técnicamente como **un producto inacabado con un propósito único y sencillo: la validación**. La metáfora que usa el material es la del **salvavidas antes del pozo sin fondo**: antes de hundirte (gastar meses y miles de dólares para descubrir que la idea no funciona), validas si tiene futuro. El prototipo es el instrumento para llegar rápido a esa conclusión —al éxito o al fracaso— de forma barata.

El curso lo diferencia explícitamente de la **Prueba de Concepto** Proof of Concept · PoC y del **Producto Mínimo Viable** Minimum Viable Product · MVP, reconociendo que "pueden tener trasfondos metodológicos distintos". Esta distinción es justo donde el curso adopta un uso particular del término, así que conviene precisarlo con cuidado.

NOTA — DISCREPANCIA 1 PROTOTIPO VS POC VS MVP

En la terminología estándar de producto, los tres son herramientas **distintas** que reducen **riesgos distintos** [confirmado]:

PoC responde "¿es esto *técnicamente posible*?" — interno, sin interfaz, datos quemados ("hard-coded"), suele desecharse. **Prototipo** (sentido clásico) responde "¿cómo se ve y se siente?" — modelo interactivo de UX/diseño, normalmente con **datos ficticios**, no apto para producción. **MVP** responde "¿lo quiere el mercado?" — un producto real y funcional, lanzado a usuarios reales.

El "prototipo de IA" del curso es en realidad un **híbrido**: valida a la vez la viabilidad técnica (territorio del PoC) y el impacto/adopción del usuario (territorio del MVP), de punta a punta, en 2–4 semanas. Y choca con el prototipo clásico en un punto fino pero examinable: el curso **exige datos realistas de producción**, mientras que un prototipo clásico usa datos ficticios. **Regla para el examen**: si te preguntan "qué es un prototipo en esta unidad", responde con la definición del curso (producto inacabado, propósito = validar impacto + viabilidad, integración vertical, datos reales, 2–4 semanas). Si te piden *distinguir* PoC/Prototipo/MVP en general, da el reparto de tres: PoC = factibilidad técnica, Prototipo = experiencia de usuario, MVP = encaje con el mercado.

2.2 Las cuatro características de un prototipo de IA

El curso caracteriza el prototipo de IA con cuatro rasgos que funcionan juntos.

(a) Integración vertical profunda. Ya definida: cubre desde la ingesta de datos hasta la interfaz, con complejidad técnica general **baja**. Esta es la tensión central y deliberada del concepto: **amplio pero superficial**. Tienes que tocar todas las estaciones del proceso, pero ninguna a fondo. ¿Por qué? Porque solo un hilo completo de extremo a extremo permite validar lo que importa: que los datos reales alimenten un modelo y que un humano vea y acepte el resultado.

(b) Temporalidad y alcance. Es un desarrollo **rápido: no más de dos a cuatro semanas, a veces solo días**. Debe tener un objetivo claro (por ejemplo, superar una línea de base específica), plazos estrictos y criterios de aceptación bien definidos. La velocidad no es un capricho: es lo que permite el **aprendizaje acelerado**. Un prototipo que tarda tres meses ya dejó de ser un prototipo y se convirtió en el proyecto que querías evitar.

(c) Realismo de datos: "no comprometerse con los datos". La regla de oro. **No comprometerse** (*don't commit to the data*) significa usar **únicamente información que estaría disponible en un escenario de producción real**, evitando limpiezas manuales que no sean escalables. La razón es contundente y vale citarla como la fórmula del curso: **el modelo y la interfaz se pueden arreglar, pero los datos no**. Si validas tu idea con datos que limpiaste a mano —rellenando huecos, corrigiendo errores que en producción seguirán llegando— estás validando una mentira: demostraste que la idea funciona en un mundo que no existe.

FIBROMIN — LA REGLA DE LOS DATOS EN CARNE PROPIA

Imagina que prototipas el predictivo de fallas. Tus sensores de vibración tienen huecos cuando se cae la red, y a veces mandan lecturas corruptas. Si para el prototipo rellenas esos huecos a mano y descartas las lecturas malas una por una, demuestras que el modelo predice fallas... con datos perfectos que **en producción nunca tendrás**. El día que escales, los huecos y la basura vuelven, la señal se degrada y el modelo que "funcionaba" empieza a fallar. Validaste un espejismo. La disciplina correcta: prototipar con los datos sucios tal como llegan, y si igual hay señal suficiente, **esa** sí es una validación real.

CONEXIÓN — UNIDAD ANTERIOR: VERACIDAD

Esta regla es la versión operativa de la **veracidad** (la "V" de Big Data que vimos como la confiabilidad del dato). "No comprometerse con los datos" es lo que protege la veracidad *dentro* de un prototipo: te obliga a validar con el nivel de confiabilidad que realmente vas a tener, no con uno inflado artificialmente.

2.3 El propósito del prototipado: minimizar el riesgo

El propósito de fondo es **minimizar el riesgo** asociado a esa increíble incertidumbre, que afecta dimensiones críticas: los requisitos, el alcance, la infraestructura, la legalidad y —sobre todo— la **calidad del resultado**, que es muy difícil de predecir hasta que se construye el modelo real. Dentro de ese propósito general, el curso lista varios propósitos específicos.

Evitar el "pozo sin fondo" *bottomless pit*. El patrón que se quiere prevenir: empresas que inician un proyecto con entusiasmo y descubren meses después, tras gastar miles de dólares, que la solución no funciona. El prototipo permite llegar a esa conclusión (o al éxito) más rápido y barato, ofreciendo mayor **retorno de la inversión** *ROI*. Es el clásico "dejar de echarle plata a un hoyo", pero detectándolo en semanas en lugar de en trimestres.

Validación de impacto y viabilidad. Confirmar si la solución aporta valor al usuario, si se usa como se previó y si la organización la acepta (impacto); y, en el frente técnico, si los datos contienen **suficientes señales** para construir un modelo útil, o si un servicio de IA externo funciona bien con la información propia de la empresa (viabilidad). Estas dos dimensiones estructuran toda la unidad y se detallan en §2.4.

Priorizar la utilidad sobre la precisión. La frase que resume la unidad: **"crear un modelo preciso es bueno, pero crear un modelo útil es mejor"**. Un modelo preciso es un logro *técnico*; un modelo útil es un logro de *negocio*. El prototipo existe para poner la solución frente a usuarios reales y obtener pronto una opinión sobre su utilidad práctica, no para exhibir métricas de exactitud que a nadie le sirven si la herramienta no se adopta.

Gestión de expectativas. Involucrar a las partes interesadas *stakeholders* desde el principio permite identificar temprano áreas problemáticas (por ejemplo, fallas en la calidad de los datos) y ajustar las expectativas sobre lo que la tecnología puede lograr de verdad. Mejor descubrir en la semana dos que "los datos no alcanzan" que prometer maravillas y desilusionar en el mes seis.

Transición hacia la producción. Un prototipo bien diseñado bajo una **arquitectura abierta** permite una transición fluida hacia las fases de prueba y producción, asegurando que la base técnica sea lo bastante sólida para escalar si la validación fue exitosa. Aquí "abierta" se opone a "encerrada": un prototipo hecho de piezas portables se puede crecer; uno hecho a la medida de una máquina aislada, no (ver "prototipado en isla", §4).

2.4 Las dos dimensiones críticas de validación

Una regla previa: **un prototipo no debe construirse si no hay algo concreto que validar**. Y lo que se valida se divide siempre en dos dimensiones.

Dimensión	Qué valida	Preguntas clave
Impacto impact	El valor de negocio y la adopción por parte del usuario.	¿Aporta valor real al usuario final? ¿Se usa como fue prevista? ¿La aceptan los usuarios dentro de su flujo de trabajo?
Viabilidad viability	La factibilidad técnica con los recursos actuales.	¿Tienen los datos suficientes señales o patrones para un modelo útil? ¿Un servicio de IA preentrenado funciona bien con los datos específicos de la empresa?

CONEXIÓN — UNIDAD ANTERIOR: CASOS CAMPEONES

Estos dos ejes **son la misma matriz 2x2 de los casos campeones** (alto impacto × alta viabilidad) que usamos para *elegir* iniciativas. La diferencia es el momento: la matriz te dice **en qué apostar** (una estimación *a priori*); el prototipo **confirma la apuesta con evidencia real** (*a posteriori*, barato). Un caso campeón es una hipótesis de "alto-alto"; el prototipo es el experimento que la pone a prueba. Por eso valida exactamente esas dos cosas y no otras.

2.5 El alcance de un prototipo efectivo

Para que el prototipo sea gestión de riesgos y no una distracción técnica, su alcance debe quedar acotado por **tres elementos**, que son las tres patas del "objetivo claro, plazos estrictos, criterios de aceptación" ya mencionados.

- Objetivo claro.** Planteado como una **pregunta de investigación** o una comparación técnica. Ejemplo del curso: *"¿Podemos construir un modelo mejor que nuestra línea de base actual para predecir el fenómeno Y con los datos X?"*. La **línea de base** *baseline* es el punto de comparación que debes superar: el sistema o regla simple que ya usas hoy. Sin línea de base, "funciona bien" no significa nada.
- Plazos estrictos.** Rápidos, para forzar el aprendizaje acelerado. Ejemplo: *"Construiremos el mejor modelo posible en un máximo de dos semanas"*. El plazo no es flexible; es parte del diseño.
- Criterios de aceptación** *acceptance criteria*. Las condiciones bajo las cuales el prototipo se considera exitoso, definidas *antes* de empezar. Ejemplo: *"El prototipo supera la prueba si genera la respuesta F, o si el usuario lo acepta tras la prueba X"*. Definirlos al final es hacer trampa: siempre encontrarás una métrica que "salió bien".

Y de remate, el alcance técnico debe garantizar **integración vertical profunda con baja complejidad técnica general**, respetando la regla de oro de no comprometerse con los datos. La frase ancla, otra vez: el modelo y la interfaz se arreglan; los datos, no.

3 Prototipos en Inteligencia de Negocio (BI)

Esta sección explica *por qué* el prototipado es un cambio de paradigma necesario y no un capricho metodológico. **Inteligencia de Negocio** *Business Intelligence* · BI es el conjunto de prácticas y herramientas para transformar datos en informes y métricas que apoyen decisiones. El argumento del curso: la forma tradicional de hacer BI ya no sirve para integrar IA.

3.1 El método de cascada en BI y su límite

Durante décadas, el estándar para desarrollar sistemas de BI fue el **método de cascada** *waterfall*: un proceso lineal y rígido que arranca con una recopilación exhaustiva de requisitos y avanza secuencialmente —diseño, ingeniería de datos, pruebas, implementación— sin volver atrás. (La sección §5 lo define a fondo.) Este modelo llegó a su límite por la convergencia de dos factores:

- **Requisitos empresariales difusos.** En un entorno cada vez más complejo, los requerimientos ya no son claros desde el día uno; **evolucionan a medida que se descubre el potencial de los datos**. Pedir que se "congelen" al inicio es pedir algo imposible.
- **Aumento de la complejidad tecnológica.** Hace una década bastaba integrar un puñado de fuentes de datos; hoy incluso empresas pequeñas gestionan docenas de sistemas y plataformas a la vez.

El golpe definitivo lo da la incertidumbre de la IA: bajo cascada, para cuando un plan a gran escala está listo, **ya fue superado por la realidad**. Con IA, ese desfase se vuelve insostenible.

3.2 El BI como un "producto de datos"

La propuesta es dejar de ver el BI como un conjunto de informes estáticos y tratarlo como un **producto de datos** *data product*: algo vivo, que se itera. Este cambio de mentalidad exige reconocer de antemano que **no se tiene certeza** sobre tres cosas antes de invertir en serio:

1. Si la idea inicial funcionará finalmente según lo previsto.
2. Qué aspecto tendrá la solución técnica definitiva.
3. Cómo interactuarán realmente los usuarios finales con la herramienta.

El prototipado es la técnica de gestión de productos ideal para mitigar esos riesgos, porque habilita un desarrollo mucho más flexible y adaptativo.

3.3 El posicionamiento estratégico del prototipo

Una distinción técnica crucial: en este enfoque, **el prototipo va ANTES de las pruebas**. Tradicionalmente las organizaciones usan **sistemas de prueba** *test systems* que intentan replicar fielmente todo el entorno de producción: el **almacén de datos** *data warehouse*, los procesos **ETL** *Extract, Transform, Load* (extraer, transformar, cargar) y un frontend de prueba. El problema es que construir esos sistemas implica **el mismo esfuerzo, costo y reto que el sistema final** — sin haber validado primero la hipótesis de valor. Es construir la réplica completa de una fábrica para recién entonces preguntarte si el producto se vende.

El prototipo se diferencia por buscar la integración vertical profunda con baja complejidad, ejecutándose en días o pocas semanas (idealmente dos a cuatro). Mientras los sistemas de BI **monolíticos** (una sola pieza grande e indivisible) no permiten cambios rápidos, un prototipo bien diseñado sí.

LA FIGURA CLAVE DE LA UNIDAD (P. 12)

El diagrama "Creación de prototipos en el proceso de desarrollo de software" ordena tres momentos de izquierda a derecha. **Creación de prototipos**: un único bloque "Prototipo" con dos flechas cruzadas — *integración vertical de extremo a extremo* (alto) y *complejidad técnica* (baja). **Testeado**: tres bloques separados (sistema de pruebas para BI, para almacén de datos, para ETL) — alta complejidad, muchas piezas. **Producción**: el "sistema de producción" como un *monolito*. La lectura: el prototipo es **barato y completo**; los sistemas de prueba y producción son **caros y fragmentados o monolíticos**. Por eso el prototipo va primero: filtra las ideas malas antes de pagar el costo de los otros dos.

3.4 Requisitos de la pila tecnológica para prototipar

Para que la experimentación sea efectiva, la **pila tecnológica** *tech stack* (el conjunto de tecnologías que usas) debe cumplir tres condiciones.

- **Soporte de las tres capas.** Debe sostener a la vez la capa de **datos** (disponibilidad), la de **análisis** (funcionamiento del modelo) y la de **interfaz de usuario** (validación del valor por un humano). Las tres capas se detallan en §4.
- **Conectividad y apertura.** Hay que **evitar el "prototipado en isla"**: experimentar en una máquina local aislada de la que no puedes salir. La plataforma debe ofrecer gran conectividad para una transición fluida a prueba y producción; los **contenedores** ayudan a esa portabilidad.
- **Uso de servicios gestionados en la nube.** La plataforma ideal da acceso inmediato a servicios de IA y ML listos para usar. Se recomienda priorizar **AutoML** para clasificación y regresión, **IA como Servicio (AlaaS)** para funciones básicas (como reconocimiento de caracteres), y **modelos preentrenados** para aplicaciones personalizadas — evitando el costo innecesario de entrenar sistemas desde cero en la etapa de validación. (Todos definidos en §4.)

4 El kit de herramientas de prototipado de IA

El kit no debe concebirse como una arquitectura definitiva e inamovible, sino como un **entorno flexible** para experimentar rápido, conectar fuentes de datos y acceder a servicios analíticos listos, manteniendo bajos los costos iniciales. El enfoque debe ser **tecnológicamente neutral**: no se trata de casarse con una marca, sino de cubrir las funciones.

Un kit completo resuelve tres preguntas: **dónde residen los datos, dónde se genera el modelo y cómo se entrega el resultado** a quien decide. Esas tres preguntas dan origen a las tres capas.

CONEXIÓN — UNIDAD 2

El propio material lo dice: estas tres preguntas son las que "estudiamos en la unidad 2". Aquí simplemente se aterrizan en una arquitectura de prototipo de tres capas. Si recuerdas el flujo dato → modelo → entrega de la unidad 2, esta sección es ese mismo flujo convertido en pila tecnológica.

4.1 Capa de datos data layer (1 de 3)

El entorno donde se **almacena, organiza y carga la información**. Maneja dos tipos:

- **Datos estructurados** structured data: organizados en filas y columnas, como tablas CSV (compras, categorías, lecturas de sensores).
- **Datos no estructurados** unstructured data: sin formato tabular — imágenes, documentos, audios, registros de texto.

Esta capa debe tener **capacidad de crecimiento flexible** (elástica) y permitir acceso rápido para pruebas iterativas, sin tener que rediseñar la infraestructura cada vez. La analogía: es la **despensa** de la cocina — debe poder agrandarse y dar acceso rápido a los ingredientes. Ejemplos: Azure Blob Storage, Amazon S3, Google Cloud Storage.

CONEXIÓN — BIG DATA (VARIEDAD)

La distinción estructurado/no estructurado es la "V" de **variedad** que vimos en Big Data. Un kit de prototipado debe soportar ambas porque la realidad de las empresas mezcla las dos: el predictivo de Fibromin combina series numéricas de sensores (estructurado) con, quizás, fotos de inspección o reportes de mantenimiento (no estructurado).

4.2 Capa de análisis analysis layer (2 de 3)

El **núcleo donde se construyen experimentos, se entrenan modelos y se evalúan resultados** — la cocina. Un kit potente ofrece tres niveles de potencia según quién lo use:

- **AutoML** Automated Machine Learning: automatiza tareas de regresión y clasificación, acelerando los prototipos iniciales incluso para perfiles con menos experiencia técnica. Es el **sous-chef** que prueba veinte recetas (modelos) por ti, las compara y te dice cuál quedó mejor — sin que tengas que cocinar cada una a mano.
- **IA como Servicio (AlaaS)** AI as a Service: servicios **preentrenados** accesibles vía API para funciones básicas como el **reconocimiento óptico de caracteres** OCR o el **procesamiento de lenguaje natural** NLP. Pides el plato ya hecho.
- **Personalización**: espacios para usuarios técnicos que necesiten entrenar modelos a medida o usar arquitecturas de **aprendizaje profundo** deep learning. Cuando ningún plato del menú sirve, cocinas tú desde cero.

CONEXIÓN — UNIDAD ANTERIOR: AIAAS VS PAAS (ES UN ESPECTRO)

Estos tres niveles son justamente el **espectro** que vimos: de "no gestionas casi nada" a "gestionas casi todo". **AlaaS** es el extremo más gestionado (modelo hecho, pago por uso). La **personalización** con plataformas como Azure ML, Vertex AI o SageMaker es el extremo **PaaS** (tú entrenas con tus datos sobre la plataforma). AutoML cae en medio. La recomendación del curso —AlaaS y preentrenados primero, modelo propio solo si hace falta— es coherente con "fallar barato": empieza por lo más gestionado y barato.

4.3 Capa de usuario user / UI layer (3 de 3)

La **interfaz de visualización o interacción que traduce las predicciones técnicas en resultados observables y comprensibles** para los usuarios finales — el plato servido en la mesa. Esta capa es vital: **un prototipo solo cumple su función cuando puede ser discutido y validado en un contexto de decisión real**. Un modelo con 95 % de exactitud que vive en un cuaderno de código que nadie del negocio puede abrir, no validó nada de su impacto. Ejemplos: Power BI, Amazon QuickSight, Looker, o frameworks ligeros como Streamlit/Dash.

CONEXIÓN — LAS DOS DIMENSIONES

Nota la simetría: la capa de **datos** y la de **análisis** sirven sobre todo para validar la **viabilidad** (¿hay señal?, ¿el modelo funciona?); la capa de **usuario** sirve para validar el **impacto** (¿el humano lo entiende, lo acepta, actúa?). Por eso las tres capas son obligatorias: te faltan herramientas para validar una de las dos dimensiones si omites alguna.

4.4 Ecosistemas tecnológicos disponibles

El mercado ofrece combinaciones equivalentes de estas tres capas. La elección entre uno y otro depende de la infraestructura previa de la empresa, los costos y el nivel de especialización técnica. **Lo importante no es dominar una marca, sino que la tecnología soporte las tres capas.**

Componente	Ecosistema AWS	Ecosistema GCP	Ecosistema Azure	Código abierto / Híbrido
Almacenamiento (capa datos)	Amazon S3	Google Cloud Storage	Azure Blob Storage	PostgreSQL / Docker
Análisis / ML (capa análisis)	Amazon SageMaker	Vertex AI	Azure Machine Learning	Python / R / MLflow
Servicios de IA (AlaaS)	AWS AI Services	Google Cloud AI APIs	Azure Cognitive Services	Modelos preentrenados
Visualización (capa usuario)	Amazon QuickSight	Looker / Looker Studio	Power BI	Streamlit / Dash

Lee la tabla por filas: cada fila es una de las tres capas (más una fila de AlaaS), y cada columna un proveedor. Las cuatro celdas de una fila son intercambiables — hacen lo mismo. **MLflow** gestiona el ciclo de vida de modelos; **Streamlit/Dash** crean interfaces de datos con poco código en Python; **PostgreSQL** es una base de datos relacional abierta.

4.5 Recomendaciones técnicas: conectividad, apertura y Docker

Sea cual sea la herramienta, la plataforma ideal debe tener **alta conectividad y ser lo bastante abierta** para una transición fluida a prueba o producción. Para reducir la fricción técnica se recomienda **Docker** y, en general, los **contenedores** *containers*: empaquetan tu flujo de trabajo para transferirlo entre entornos (de una máquina local a un servidor remoto) sin que cambie la hipótesis de validación.

La analogía del contenedor: es el **tupper hermético** que te permite llevar exactamente el mismo plato de una cocina a otra sin que se desarme ni cambie de sabor. "Funciona en mi máquina" deja de ser excusa porque el contenedor lleva consigo todo lo que el programa necesita. Es, además, el antídoto directo contra el **prototipado en isla**: lo que va en un contenedor, sale de la isla.

5 Agilidad vs. Cascada en el prototipado de IA

Esta sección formaliza las dos grandes corrientes de gestión de proyectos para explicar por qué la agilidad encaja con la IA y la cascada no.

5.1 Definición de los dos modelos

Metodología en cascada Waterfall	Metodología ágil Agile
Modelo de ciclo de vida secuencial y lineal . Fases rígidas y separadas: recopilación de requisitos → diseño → implementación → verificación → mantenimiento. Una fase no inicia hasta que la anterior se completa y aprueba por completo. Asume que el sistema es totalmente especificable y predecible desde el inicio , por lo que exige planificación meticulosa y extensa por adelantado.	No es un proceso paso a paso, sino una filosofía de ciclos iterativos e incrementales llamados sprints . En vez de planificación rígida inicial, se basa en colaboración continua, adaptabilidad y entrega rápida de incrementos funcionales. Prioriza individuos e interacciones sobre procesos y herramientas , buscando satisfacer al cliente con entrega temprana y continua de valor.

La analogía: la cascada es construir un **crucero completo según un plano congelado** y recién botarlo al agua al final — si flota mal, ya gastaste todo. La agilidad es construir un **bote, probarlo, agrandarlo, probarlo de nuevo**: cada vuelta entrega algo que flota y enseña algo. (Es, dicho sea de paso, tu propia máxima de *iterar en vueltas cortas* aplicada a la ingeniería de software.)

5.2 Limitaciones de la cascada frente a la incertidumbre de la IA

La barrera principal es la increíble incertidumbre del ML: es muy difícil predecir de antemano el alcance, la infraestructura necesaria o si los datos contienen las "señales" suficientes. Si un equipo usa cascada para IA, se ve obligado a **"congelar" los requisitos** al inicio. Y si meses después, en implementación o pruebas, descubre que los datos no eran adecuados o que la precisión es insuficiente, **el costo de volver atrás es astronómico**.

El nombre técnico del riesgo central es el **feedback tardío** late feedback: los clientes solo ven el producto final en las etapas finales, cuando ya es difícil hacer ajustes significativos. La analogía: es cocinar un banquete de cinco platos **sin probar nada hasta servirlo**; si está salado, ya no hay vuelta. La cascada es eficiente cuando el plato es conocido y la receta es fija; es un desastre cuando estás inventando la receta sobre la marcha — que es precisamente la situación de la IA.

5.3 Ventajas de la agilidad para el prototipado

Tratando la solución como "producto de datos" y aceptando que no se sabe de antemano si la idea funcionará, la agilidad ofrece cinco ventajas:

- **Validación de viabilidad temprana.** Acepta que el problema no puede definirse del todo al inicio; en iteraciones de 2–4 semanas valida pronto si un servicio de IA funciona con los datos propios antes de escalar.
- **Mentalidad de "fallar rápido y barato".** Con prototipos ágiles —productos inacabados solo para validar— se evita el "pozo sin fondo". Si una idea no funciona, se detecta en días.
- **Flexibilidad ante requisitos emergentes.** Mientras la cascada *resiste* el cambio, la agilidad lo *aprovecha*. En IA es común que el análisis de datos arroje hallazgos que obliguen a cambiar de rumbo; el enfoque ágil permite **pivotar** pivot (girar de estrategia) sin reiniciar todo el proyecto.
- **Interacción continua con usuarios reales.** La única forma de saber si un modelo de ML es útil es ponerlo pronto frente a usuarios reales; la agilidad fomenta esa retroalimentación frecuente.
- **Reducción de riesgos técnicos.** Al integrar codificación y pruebas en cada ciclo (en vez de fases separadas), se detectan errores de diseño y arquitectura de inmediato, evitando fallos costosos al final.

Conclusión del curso: para contextos de alta ambigüedad como la IA, lo ágil es una **propuesta de superación** de lo lineal — permite transitar del descubrimiento a la validación con una adaptabilidad que la cascada, pensada para entornos predecibles, no puede ofrecer.

6 Caso de estudio: Comercial Luma

El curso aterriza todo en una empresa chilena ficticia. Vale la pena seguirlo paso a paso porque es exactamente cómo se ve la teoría en acción — y porque es el tipo de caso que cae en el examen.

Etapa del caso	Qué pasó y qué concepto ilustra
El problema	Comercial Luma (venta online de hogar, tecnología y cuidado personal) ve que muchos clientes navegan y abandonan sin comprar . Sospechan que la causa es la baja personalización : la home muestra "los más vendidos de la semana", sin considerar historial ni compras previas.
La propuesta	La gerencia quiere un sistema de recomendaciones con IA. Pero antes de invertir en la solución completa, el equipo de datos sugiere un prototipo para validar si la idea genera valor. Valentina Rojas (analista de BI) coordina y su primera decisión es acotar el alcance : un prototipo funcional en dos semanas , solo para validar.
El objetivo claro	Una pregunta concreta: <i>¿los datos históricos de navegación y compra permiten generar recomendaciones más útiles que el sistema actual de productos populares?</i> — Es exactamente un objetivo planteado como pregunta de investigación, con línea de base = el sistema de "más vendidos".
Los datos (realismo)	Reúnen una muestra representativa : clics, productos vistos, frecuencia de compra, categorías, compras previas. La cargan en un entorno temporal evitando limpiezas manuales excesivas que luego no se puedan replicar en producción. (La regla de oro en acción.)
Los tres problemas	(1) Datos de navegación incompletos en algunos períodos (el sitio no siempre registró usuarios no autenticados). (2) Marketing quiere resultados rápidos pero no tiene claro el criterio de éxito . (3) Tecnología advierte que no conviene una arquitectura demasiado compleja sin saber si la idea funcionará. Además, tensión entre áreas: marketing quiere interfaz bonita; tecnología quiere validar viabilidad. Valentina recuerda: el prototipo produce evidencia temprana , no es el producto final.
Criterios de aceptación	El prototipo es exitoso si cumple tres condiciones : el modelo supera la recomendación actual; los usuarios del piloto muestran más interés en las sugerencias personalizadas; y los datos se pueden obtener igual en producción . (Impacto + viabilidad + realismo, hechos criterio.)
Las tres capas	Datos : muestra histórica estructurada (compras, categorías). Análisis : una herramienta de AutoML para probar varios modelos y compararlos con la línea de base. Usuario : una interfaz simple que muestra las recomendaciones a un grupo reducido.
La prueba (piloto)	El grupo piloto recibe dos listas: populares vs. personalizada. Comparan clics, productos guardados y comentarios. Aunque el modelo no es perfecto, las personalizadas logran más interacción.
El cierre	Valentina presenta a gerencia: el prototipo no se evalúa por su sofisticación, sino por su capacidad de validar impacto y viabilidad . Mostró señales positivas, generó interés e identificó mejoras necesarias <i>antes</i> de escalar. Validar con datos reales, usuarios concretos y criterios claros permitió decidir si valía la pena avanzar.

FIBROMIN — EL MISMO CASO, TU INDUSTRIA

Traduce Luma a Fibromin y tienes tu propio playbook. **Problema**: paradas no planificadas de equipos. **Línea de base**: la regla actual ("cambiar la pieza cada X horas" o esperar a que falle). **Objetivo claro**: "¿los datos de sensores predicen la falla con suficiente anticipación como para superar la regla actual?". **Tres capas**: Blob/S3 con históricos de vibración y temperatura (datos) → AutoML probando modelos de predicción de falla (análisis) → una pantalla simple que le muestra al jefe de mantención "equipo 7: riesgo alto en 48 h" (usuario). **Criterios de aceptación**: el modelo anticipa más fallas reales que la regla actual, el mantenedor confía y actúa, y los datos se obtienen igual en producción. Dos semanas, datos sucios reales, una pantalla fea pero funcional. Si hay señal, escalas; si no, te ahorraste el pozo sin fondo. **Y recuerda**: este predictivo ya era tu "caso campeón" de la unidad anterior — el prototipo es lo que confirma esa apuesta antes de gastar.

7 Aplicación práctica: aseguradora con IA generativa

El ejercicio de cierre plantea otro escenario para que *tú* apliques el método: una aseguradora que recibe miles de denuncias de siniestros al mes y evalúa una solución de **IA generativa** *generative AI* que ayude a los analistas a revisar documentación, resumir antecedentes y sugerir cursos de acción. No introduce conceptos nuevos; es un simulacro de aplicación. Te dejo el esqueleto de respuesta para que veas cómo se conecta todo (estas son las respuestas que el examen busca).

Bloque del ejercicio	Cómo responderlo con los conceptos de la unidad
1. Definición del prototipo	Hipótesis: "un modelo de IA generativa puede resumir denuncias y sugerir acciones con calidad suficiente para acelerar al analista". Pregunta de negocio: ¿reduce el tiempo de revisión sin perder precisión? Objetivo acotado: en 2–3 semanas, procesar una muestra real de denuncias y medir si los resúmenes son útiles para los analistas frente a su proceso manual (línea de base).
2. Alcance y criterios	Mínimo: ingerir documentos reales, generar resumen + sugerencia, mostrarlos en una interfaz simple. Fuera: integración con sistemas core, seguridad de producción, UI pulida, automatización total. Éxito: los analistas califican los resúmenes como útiles y el tiempo de revisión baja respecto al manual.
3. Datos y viabilidad	Necesitas: denuncias reales con su documentación (texto no estructurado). Por qué datos reales: si limpias o eliges casos fáciles, validas un mundo que no existe (regla de oro). Riesgo de datos "mejorados": sobreestimas la calidad; en producción la documentación llega desordenada y el modelo se cae.
4. Herramientas y arquitectura	Capas: almacenamiento de documentos (datos) → un modelo de IA generativa preentrenado vía API / AlaaS (análisis) → interfaz simple para el analista (usuario). Por qué preentrenado/externo: entrenar un generativo propio en la fase de validación es caro e innecesario; "fallar barato" exige empezar por lo gestionado. Ventaja de bajo costo: validas la idea en semanas sin construir infraestructura definitiva.
5. Metodología	Ágil > cascada porque los requisitos de "qué resumen es útil" emergen al ver resultados reales; congelarlos al inicio es imposible. Retroalimentación temprana: los analistas reales dicen pronto si sirve. Reduce riesgo de inversión: detectas un fracaso en semanas, no en meses.
Reflexión final	Evidencia para avanzar: ahorro de tiempo medible + aceptación de los analistas + datos replicables. Señales para replantear/abandonar: los resúmenes inducen a error, los analistas no confían, o la documentación real es demasiado mala. Por qué sofisticado ≠ valor: un generativo impresionante que los analistas no adoptan no genera valor — utilidad > precisión, otra vez.

8 Síntesis: las ideas que no se te pueden olvidar

Si tuvieras que llevarte la unidad en una frase: **prototipar no es construir la arquitectura final, sino aprender —rápido y barato— si una idea de IA genera valor real antes de escalarla o descartarla**. De ahí cuelga todo lo demás.

El **prototipo** es un producto inacabado cuyo único fin es **validar** dos dimensiones: **impacto** (valor y adopción) y **viabilidad** (¿hay señal en los datos?, ¿funciona el modelo?) — las mismas dos del caso campeón, ahora puestas a prueba. Para que sirva debe tener **integración vertical profunda** (de los datos a la interfaz) con **baja complejidad**, en **2–4 semanas**, acotado por **objetivo claro, plazos estrictos y criterios de aceptación**, y respetando la regla de oro: **no comprometerse con los datos** (el modelo y la interfaz se arreglan; los datos, no).

Técnicamente se apoya en una **arquitectura de tres capas** —datos, análisis, usuario— montada sobre **servicios gestionados en la nube** (AutoML, AlaaS, modelos preentrenados), evitando el **prototipado en isla** y usando **contenedores** para la portabilidad. Y se ejecuta con mentalidad **ágil**, no en **cascada**, porque la incertidumbre de la IA hace insostenible congelar requisitos y esperar al feedback tardío. Comercial Luma —y tu Fibromin— muestran el método completo en acción: dos semanas, datos reales, una pantalla simple, criterios claros, decisión informada.

Notas de discrepancia con la práctica actual

Dos puntos donde la literatura o el estado actual de los productos matizan o actualizan lo que dice el curso. En ambos, prioriza la versión del curso para el examen y ten la otra como contexto.

DISCREPANCIA 1 "PROTOTIPO" DEL CURSO VS. POC / PROTOTIPO / MVP ESTÁNDAR

El curso dice: el prototipo de IA es un producto inacabado cuyo único fin es validar impacto y viabilidad, de punta a punta, con datos reales, en 2–4 semanas; es distinto de PoC y MVP.

La literatura dice [confirmado]: PoC, prototipo y MVP son tres herramientas distintas para riesgos distintos. **PoC** = ¿es técnicamente posible? (interno, datos quemados, desechable). **Prototipo** (clásico) = ¿cómo se ve y se siente? (UX/diseño, datos ficticios, no productivo). **MVP** = ¿lo quiere el mercado? (producto real lanzado a usuarios). El "prototipo" del curso es un **híbrido PoC+MVP** y, además, contradice al prototipo clásico en un punto: exige datos reales donde el clásico usa ficticios.

Regla práctica: "¿Qué es un prototipo en esta unidad?" → definición del curso. "Distingue PoC, prototipo y MVP" → reparto de tres (factibilidad / experiencia / mercado).

DISCREPANCIA 2 AZURE MACHINE LEARNING STUDIO

El curso dice: "Azure Machine Learning Studio: interfaz web para entrenar e implementar modelos sin código".

El estado actual dice [confirmado]: el producto sin código que encajaba con esa frase, **"Azure ML Studio (classic)", fue retirado el 31 de agosto de 2024**. Hoy "Azure Machine Learning studio" es un entorno unificado **principalmente de código**, y la experiencia visual sin código la entrega el **"Azure ML designer"**.

Regla práctica: para el examen, vale la definición del curso. En la vida real, "interfaz sin código de Azure" = el *designer*; el *Studio classic* ya no existe.

Glosario de la Unidad 4

Todos los términos técnicos del módulo. La columna "Inglés" trae el término cuando el inglés es el estándar del campo.

Término	Inglés	Definición en una línea
Prototipo	Prototype	Producto inacabado cuyo único propósito es validar hipótesis de negocio y técnicas.
Prueba de Concepto	Proof of Concept (PoC)	Experimento que prueba si una idea es técnicamente posible, antes de comprometer recursos.
Producto Mínimo Viable	Minimum Viable Product (MVP)	Producto real y funcional con lo mínimo para validar el encaje con el mercado ante usuarios reales.
Incertidumbre	Uncertainty	Falta de previsibilidad sobre requisitos, aceptación del usuario y calidad de los datos en proyectos de ML.
Inteligencia de Negocio	Business Intelligence (BI)	Prácticas y herramientas para convertir datos en informes y métricas que apoyan decisiones.
Producto de datos	Data product	Tratar el BI como algo vivo e iterable, no como informes estáticos.
Impacto	Impact	Dimensión de validación centrada en el valor de negocio y la adopción por el usuario.
Viabilidad	Viability / Feasibility	Dimensión de validación centrada en la factibilidad técnica con los recursos actuales.
Criterios de aceptación	Acceptance criteria	Condiciones, definidas de antemano, bajo las cuales el prototipo se considera exitoso.
Línea de base	Baseline	Punto de comparación (sistema o regla actual) que el prototipo debe superar.
Señales en los datos	Signals / patterns	Patrones suficientes en los datos para que un modelo sea realmente útil.
Utilidad sobre precisión	Utility over accuracy	Principio: un modelo útil (logro de negocio) vale más que uno solo preciso (logro técnico).
Retorno de la inversión	Return on Investment (ROI)	Relación entre el valor obtenido y lo invertido; el prototipo lo mejora al fallar barato.
Pozo sin fondo	Bottomless pit	Proyecto que consume recursos indefinidamente sin entregar valor; el prototipo lo previene.
Gestión de expectativas	Expectation management	Involucrar a las partes interesadas temprano para ajustar lo que la tecnología puede lograr.
Integración vertical profunda	Deep vertical integration	Cubrir todas las capas (datos a interfaz) manteniendo baja la complejidad técnica.
Realismo de datos / No comprometerse con los datos	Data realism / Don't commit to the data	Usar solo datos disponibles en producción, sin limpiezas manuales no escalables.
Objetivo claro	Clear objective	Meta del prototipo planteada como pregunta de investigación o comparación técnica.
Plazos estrictos	Strict timelines	Límite de tiempo corto (2–4 semanas) que fuerza el aprendizaje acelerado.
Baja complejidad técnica	Low technical complexity	Mantener el prototipo simple en cada capa, pese a cubrirlas todas.
Metodología en cascada	Waterfall	Modelo lineal y secuencial de fases rígidas que asume el sistema predecible desde el inicio.
Metodología ágil	Agile	Filosofía de ciclos iterativos (sprints), colaboración continua y entrega temprana de valor.
Sprint	Sprint	Ciclo corto de desarrollo iterativo e incremental dentro de la metodología ágil.
Fallar rápido y barato	Fail fast & cheap	Mentalidad de detectar pronto y a bajo costo si una idea

		no funciona.
Feedback tardío	Late feedback	Riesgo de la cascada: el cliente ve el producto solo al final, cuando el ajuste ya es caro.
Congelar requisitos	Freezing requirements	Fijar los requerimientos al inicio; inviable bajo la incertidumbre de la IA.
Pivotar	Pivot	Cambiar de estrategia ante hallazgos nuevos sin reiniciar todo el proyecto.
Capa de datos	Data layer	Entorno elástico donde se almacena, organiza y carga la información del prototipo.
Capa de análisis	Analysis layer	Núcleo donde se construyen experimentos, se entrenan y se evalúan los modelos.
Capa de usuario / interfaz	User / UI layer	Interfaz que traduce las predicciones en resultados comprensibles para validar en contexto real.
Datos estructurados	Structured data	Información en filas y columnas, como tablas CSV.
Datos no estructurados	Unstructured data	Información sin formato tabular: imágenes, documentos, audios, texto.
Prototipado en isla	Island prototyping	Experimentar en una máquina local aislada, sin ruta a prueba o producción; debe evitarse.
AutoML	Automated Machine Learning	Automatización de tareas de regresión y clasificación, para prototipar rápido sin gran expertise.
IA como Servicio	AI as a Service (AIaaS)	Servicios de IA preentrenados accesibles vía API para funciones básicas.
Reconocimiento óptico de caracteres	Optical Character Recognition (OCR)	Función de IA que extrae texto desde imágenes o documentos escaneados.
Procesamiento de lenguaje natural	Natural Language Processing (NLP)	Función de IA para entender y procesar lenguaje humano.
Modelos preentrenados	Pretrained models	Modelos ya entrenados, reutilizables para aplicaciones personalizadas sin entrenar desde cero.
Contenedores / Docker	Containers / Docker	Empaquetan un flujo de trabajo para moverlo entre entornos sin que cambie su comportamiento.
Servicios gestionados en la nube	Managed cloud services	Servicios de IA/ML listos para usar que reducen costo e infraestructura en la validación.
Instancia de cálculo	Compute instance	Máquina virtual en la nube donde se ejecuta la carga real de los modelos.
Aprendizaje profundo	Deep learning	Arquitecturas de redes neuronales profundas para tareas que requieren modelos a medida.
Azure Blob Storage	Azure Blob Storage	Almacenamiento de objetos binarios grandes (CSV, imágenes, resultados) en Azure.
Azure Machine Learning Studio	Azure ML Studio	Interfaz web de Azure para entrenar/implementar modelos sin código (ver Discrepancia 2).
Almacén de datos	Data warehouse	Repositorio central de datos integrados que los sistemas de prueba intentan replicar.
Proceso ETL	Extract, Transform, Load	Extraer, transformar y cargar datos entre sistemas; parte del entorno de producción.
Ecosistemas tecnológicos	AWS / GCP / Azure / Open Source	Conjuntos equivalentes de herramientas para cubrir las tres capas; la elección depende del contexto.

Total: 47 términos. (Este conteo coincide con las 47 flashcards del archivo interactivo de la Etapa 2.)

Mapa conceptual

Árbol jerárquico de la unidad. Las flechas simples (→) indican la relación entre nodos; los enlaces cruzados con doble flecha (==>) marcan relaciones reales no jerárquicas entre ramas distintas. Este mismo árbol es el mapa interactivo de la Etapa 2.

PROTOTIPADO DE IA – validar rápido y barato antes de invertir

- └ 1. **¿QUÉ ES UN PROTOTIPO?** (fundamentos)
 - └ Prototipo = producto inacabado, propósito único: VALIDAR
 - └ ≠ Prueba de Concepto (PoC) → PoC valida factibilidad técnica
 - └ ≠ Producto Mínimo Viable (MVP) → MVP valida encaje con el mercado
 - └ el prototipo del curso = híbrido PoC+MVP (!) (ver Discrepancia 1)
- └ 2. **¿POR QUÉ PROTOTIPAR?** (validación / motivos)
 - └ nace de la INCERTIDUMBRE → no sabes si hay señal hasta construir
 - └ evita el POZO SIN FONDO → mejora el ROI
 - └ prioriza UTILIDAD sobre PRECISIÓN ("útil > preciso")
 - └ gestiona EXPECTATIVAS → involucra stakeholders temprano
 - └ valida DOS DIMENSIONES:
 - └ IMPACTO → valor de negocio + adopción del usuario
 - └ VIABILIDAD → ¿hay señales en los datos? ¿funciona el modelo?
- └ 3. **CÓMO SE ACOTA** (alcance / reglas)
 - └ INTEGRACIÓN VERTICAL PROFUNDA → de datos a interfaz, completo
 - └ BAJA COMPLEJIDAD TÉCNICA → amplio pero superficial
 - └ TEMPORALIDAD: 2–4 semanas (a veces días)
 - └ se acota con: OBJETIVO CLARO + PLAZOS ESTRICTOS + CRITERIOS DE ACEPTACIÓN
 - └ objetivo claro se mide contra una LÍNEA DE BASE
 - └ REGLA DE ORO: no comprometerse con los datos
 - └ "el modelo y la interfaz se arreglan; los datos, no"
- └ 4. **METODOLOGÍA** (ágil vs cascada)
 - └ tratar el BI como PRODUCTO DE DATOS → asumir que no hay certeza
 - └ CASCADA (Waterfall) → lineal, rígida, congela requisitos
 - └ falla por FEEDBACK TARDÍO → ajustar al final es carísimo
 - └ ÁGIL (Agile) → sprints, iterativo, pivota
 - └ permite FALLAR RÁPIDO Y BARATO + interacción con usuarios
- └ 5. **ARQUITECTURA: TRES CAPAS** (kit de herramientas)
 - └ responde 3 preguntas: ¿dónde los datos? ¿dónde el modelo? ¿cómo se entrega? (Unidad 2)
 - └ CAPA DE DATOS → estructurados (CSV) + no estructurados (img/audio/texto)
 - └ CAPA DE ANÁLISIS → donde se entrena y evalúa
 - └ CAPA DE USUARIO → interfaz, traduce predicción a decisión
 - └ evitar PROTOTIPADO EN ISLA → máquina local sin salida
 - └ usar CONTENEDORES / DOCKER → portabilidad entre entornos
- └ 6. **HERRAMIENTAS Y SERVICIOS** (capa de análisis, en detalle)
 - └ AutoML → automatiza clasificación/regresión
 - └ AIaaS → preentrenado vía API (OCR, NLP)
 - └ MODELOS PREENTRENADOS → reutilizar sin entrenar desde cero
 - └ PERSONALIZACIÓN → aprendizaje profundo, modelos a medida
 - └ INSTANCIA DE CÁLCULO → VM en la nube que ejecuta el trabajo
- └ 7. **ECOSISTEMAS** (marcas equivalentes – gris)
 - └ AWS → S3 · SageMaker · AI Services · QuickSight
 - └ GCP → Cloud Storage · Vertex AI · Cloud AI APIs · Looker
 - └ AZURE → Blob Storage · Azure ML (Studio, ver Discrepancia 2) · Cognitive Services · Power BI
 - └ OPEN SOURCE → PostgreSQL/Docker · Python/R/MLflow · Streamlit/Dash
- └ 8. **CASO COMERCIAL LUMA** (todo aplicado)
 - └ abandono de compras → prototipo de recomendaciones en 2 semanas
 - └ con AutoML + grupo piloto → personalizadas ganan → decidir si escalar

ENLACES CRUZADOS (relaciones no jerárquicas reales):

- IMPACTO ==> CAPA DE USUARIO (el impacto se valida en la interfaz)
- VIABILIDAD ==> CAPA DE DATOS+ANÁLISIS (la viabilidad = señal + modelo)
- REALISMO DE DATOS ==> CAPA DE DATOS (datos reales, sin limpiar a mano)
- AutoML ==> CAPA DE ANÁLISIS (es la herramienta estrella de esa capa)
- ÁGIL ==> TEMPORALIDAD 2–4 sem (los sprints fijan el plazo corto)
- DOS DIMENSIONES ==> CASOS CAMPEONES (Unidad 3) (mismos ejes: impacto × viabilidad)
- AIaaS ==> ESPECTRO AIaaS<->PaaS (Unidad 3) (extremo más gestionado)

Referencias del curso: Zwingmann, T. (2024), *AI-Powered Business Intelligence*, O'Reilly Media (fuente principal de los conceptos de prototipado, integración vertical y "no comprometerse con los datos"). Calvo & Escapa (2025), *The AI Driven Business*. Kniberg (2007), *Scrum and XP from the Trenches* (base de la metodología ágil).